

VRUM MOBILE: GESTÃO E MONITORAMENTO DO TRANSPORTE ESCOLAR

Anderson Labra Villarroel ⁽¹⁾, Gustavo Batistela Blois ⁽²⁾, Matheus Santos Cerqueira ⁽³⁾. Orientador: Prof. Me. Jose Picovsky ⁽¹⁾ 5-ADS-00356200, ⁽²⁾ 5-ADS-00354456, ⁽³⁾ 5-ADS-00355689.

RESUMO

Este artigo apresenta a continuidade e a evolução técnica do aplicativo VRUM, voltado à gestão e monitoramento do transporte escolar urbano. Após a consolidação da modelagem de dados no período anterior, a etapa atual concentra-se no desenvolvimento da aplicação móvel (front-end) e na implementação do servidor (back-end). O desenvolvimento móvel foi executado em ambiente Node.js utilizando TypeScript e o framework React Native com Expo, resultando na criação de interfaces funcionais como a tela de Login. O back-end foi estruturado em Node.js com TypeScript e a biblioteca Express para a criação da API REST, incorporando criptografia de senhas, validação de segurança via código por e-mail e autenticação baseada em JSON Web Token (JWT). A integração desses componentes viabiliza o fluxo seguro de dados e aproxima o sistema de um Produto Mínimo Viável (MVP). Os resultados parciais demonstram a viabilidade técnica da arquitetura escolhida para atender com segurança e desempenho às demandas de motoristas e responsáveis.

Palavras-Chave: transporte escolar; desenvolvimento mobile; React Native; API REST; segurança da informação.

1. Introdução

O transporte escolar urbano desempenha papel fundamental na rotina de famílias e estudantes, mas enfrenta desafios de comunicação, controle de pagamentos e previsibilidade. A falta de ferramentas digitais adequadas para motoristas autônomos (perueiros) e responsáveis legais gera insegurança e falhas logísticas [1]. Para mitigar esse cenário, o projeto VRUM propõe uma solução digital integrada.

Enquanto as etapas anteriores focaram na concepção teórica e modelagem de banco de dados, o objetivo geral deste semestre consiste na implementação prática da interface mobile e do back-end do sistema. Como objetivos específicos, estabelecem-se o desenvolvimento de telas funcionais utilizando React Native e Expo, a construção de uma API modular com Express para gerenciamento de requisições, e a aplicação de mecanismos robustos de segurança, incluindo a autenticação via JWT, criptografia de credenciais e validação por e-mail. O público-alvo permanece composto por condutores e responsáveis por estudantes, visando proporcionar uma plataforma confiável e eficiente com aplicabilidade no mercado de pequenas e médias soluções logísticas.

2. Metodologia

A metodologia adotada compreende o desenvolvimento iterativo da aplicação, dividida em duas frentes complementares: front-end móvel e back-end. Para a construção do aplicativo móvel, selecionou-se o ecossistema Expo integrado ao React Native, utilizando a linguagem TypeScript sobre o ambiente de execução Node.js. Essa escolha justifica-se pela agilidade no ciclo de desenvolvimento e pela capacidade de gerar código nativo multiplataforma de forma eficiente.

No desenvolvimento do back-end, empregou-se o Node.js com TypeScript e o framework Express, ideal para a criação de APIs ágeis e modulares. O processo metodológico seguiu as etapas de: (a) especificação técnica das rotas da API; (b) codificação dos serviços de autenticação e segurança; (c) codificação das interfaces gráficas baseadas nos protótipos previamente desenhados; e (d) testes preliminares de validação de dados e segurança das credenciais. O gerenciamento de código e a colaboração foram mantidos por meio de versionamento via GitHub.

3. Desenvolvimento

O desenvolvimento do projeto nesta etapa concentrou-se na construção da base funcional do sistema, com foco em autenticação, segurança e interface do usuário.

3.1 Desenvolvimento do Front-end

O front-end do aplicativo foi desenvolvido utilizando React Native [2] com Expo e TypeScript. Essa escolha permitiu maior produtividade e facilidade na criação de interfaces modernas e responsivas.

Foi implementada a tela de login do sistema, responsável por permitir o acesso dos usuários por meio de credenciais previamente cadastradas. A interface foi projetada visando simplicidade e usabilidade, facilitando a interação do usuário.

3.2 Desenvolvimento do Back-end

O back-end foi desenvolvido utilizando Node.js com TypeScript e o framework Express [3], responsável pela criação da API da aplicação.

Para a persistência e manipulação dos dados, foi utilizado ORM Prisma, que facilita a integração com o banco de dados e otimiza operações de registro, consulta e atualização de informações.

Foram implementadas funcionalidades básicas de um sistema CRUD, incluindo cadastro de usuários, autenticação (login) e validação de dados.

Além disso, foram incorporados mecanismos de segurança, como: (a) criptografia de senhas utilizando a biblioteca Bcrypt, garantindo que dados sensíveis não sejam armazenados em texto puro; (b) autenticação via JSON Web Token (JWT), permitindo controle seguro de sessões; (c) envio de códigos de verificação por e-mail utilizando a biblioteca Nodemailer.

Essas implementações garantem maior confiabilidade e proteção dos dados dos usuários.

3.3 Segurança e Autenticação

Nesta etapa, foram implementados mecanismos de segurança fundamentais para a proteção dos dados dos usuários.

As senhas são armazenadas utilizando criptografia por meio da biblioteca Bcrypt, impedindo que informações sensíveis sejam expostas em texto puro. Além disso, foi implementado um sistema de autenticação baseado em JSON Web Token (JWT), permitindo que o usuário permaneça autenticado de forma segura após o login.

Também foi desenvolvido um sistema de validação por e-mail, no qual o usuário recebe um código de verificação durante o processo de cadastro. Esse envio é realizado automaticamente por meio da biblioteca Nodemailer, garantindo maior confiabilidade no processo.

Esses mecanismos adicionam camadas importantes de segurança, reduzindo riscos de acesso indevido e fortalecendo a proteção das informações dos usuários.

3.4 Fluxo de Autenticação

O fluxo de autenticação do sistema foi estruturado em múltiplas etapas para garantir segurança e consistência dos dados.

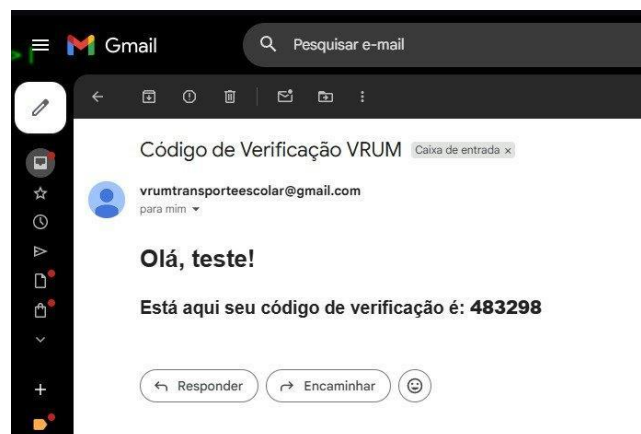
Inicialmente, o usuário realiza o cadastro informando seus dados básicos. Após essa etapa, o sistema envia um código de verificação para o e-mail informado. O usuário deve inserir esse código para validar sua conta.

Conforme ilustrado na Figura 1, o envio do código de verificação é realizado de forma automatizada, garantindo uma etapa adicional de validação durante o processo de cadastro.

Após a validação, o usuário pode realizar o login utilizando suas credenciais. Nesse momento, o sistema verifica os dados informados e, caso estejam corretos, gera um token JWT que será utilizado para autenticar as requisições futuras.

Esse fluxo garante maior segurança no acesso ao sistema, além de permitir controle eficiente das sessões de usuário.

Figura 1. Envio de código de verificação por e-mail



Fonte: autores (2026)

3.5 Arquitetura da Aplicação

A arquitetura do sistema foi estruturada com separação entre front-end e back-end, seguindo o modelo de aplicações modernas baseadas em API.

O front-end, desenvolvido em React Native com Expo, é responsável pela interface do usuário e interação com o sistema. Já o back-end, desenvolvido com Node.js e Express, gerencia as regras de negócio, autenticação e processamento das requisições.

A comunicação entre as camadas ocorre por meio de requisições HTTP, utilizando um padrão REST, permitindo escalabilidade e flexibilidade para futuras integrações.

3.6 Interfaces do Sistema

A interface do sistema foi desenvolvida com foco na usabilidade e simplicidade.

Conforme apresentado na Figura 2, a tela de login permite que o usuário acesse o sistema de forma rápida e intuitiva, utilizando e-mail e senha. O layout foi projetado para dispositivos móveis, garantindo responsividade e boa experiência de uso.

Figura 2. Tela de Login



Fonte: autores (2026)

3.7 Requisitos do Sistema

Com base na evolução do projeto e nas funcionalidades implementadas nesta etapa, foram atualizados os requisitos funcionais (Tabela 1) e não funcionais (Tabela 2) do sistema.

Tabela 1. Requisitos Funcionais do VRUM

Identificador	Requisito Funcional	Descrição
RF01	Cadastro de usuário	Permite o registro de novos usuários com validação de dados
RF02	Validação por e-mail	Envia código de verificação para confirmação de cadastro
RF03	Login com autenticação JWT	Permite acesso seguro ao sistema com geração de token
RF04	Criptografia de senha	Garante armazenamento seguro das credenciais

RF05	Redefinição de senha	Permite recuperação de acesso ao sistema
RF06	Gerenciamento de sessão	Controla o acesso do usuário por meio de token
RF07	Integração com API	Permite comunicação entre front-end e back-end
RF08	Validação de dados	Verifica integridade das informações inseridas
RF09	Estrutura de usuários	Permite organização e armazenamento de usuários
RF10	Base para futuras funcionalidades	Estrutura preparada para rotas, pagamentos e notificações

Fonte: autores (2026)

Tabela 2. Requisitos Não Funcionais do VRUM

Identificador	Requisito Não Funcional	Descrição
RNF01	Segurança de dados	Utilização de criptografia para proteção de informações
RNF02	Autenticação segura	Uso de JWT para controle de acesso
RNF03	Desempenho	Respostas do sistema em tempo adequado
RNF04	Escalabilidade	Arquitetura preparada para expansão
RNF05	Responsividade	Interface adaptável a diferentes dispositivos
RNF06	Disponibilidade	Sistema disponível para acesso contínuo
RNF07	Manutenibilidade	Código estruturado para fácil manutenção
RNF08	Integração	Capacidade de comunicação entre serviços
RNF09	Confiabilidade	Garantia de funcionamento consistente
RNF10	Compatibilidade	Suporte aos principais ambientes mobile

Fonte: autores (2026)

4. Considerações Finais

A evolução do projeto VRUM neste semestre consolidou a transição da arquitetura conceitual para uma aplicação prática funcional. A implementação do front-end mobile em React Native e do back-end em Node.js com Express demonstrou a viabilidade técnica do ecossistema JavaScript/TypeScript para a criação de sistemas robustos e seguros. A inclusão de criptografia de senhas, validação por e-mail e autenticação com JWT atende às exigências de segurança essenciais para lidar com dados de motoristas e menores de idade. Como trabalhos futuros, pretende-se a expansão das demais telas do MVP, o refinamento da integração total entre o aplicativo e os

procedimentos armazenados no banco de dados e a realização de testes de carga para avaliar o desempenho do servidor sob múltiplas requisições simultâneas.

5. Referências

[1] ESTADÃO. **Summit Mobilidade: transporte escolar no Brasil: um desafio de infraestrutura.** [acesso em 17 jun 2026]. Disponível em: <https://summitmobilidade.estadao.com.br/compartilhando-o-caminho/transporte-escolar-no-brasil-um-desafio-de-infraestrutura/>

[2] UBISTART. **Entenda o que é React Native e por onde começar.** [acesso em 02 mai 2026]. Disponível em: <https://ubistart.com/blog/entenda-o-que-e-react-native-e-por-onde-comecar/>

[3] TREINAWEB. **O que é o Express.js.** [acesso em 05 mai 2026]. Disponível em: <https://www.treinaweb.com.br/blog/o-que-e-o-express-js>